

CPE231 Algorithm Design

Laboratory 2: Experimental Performance Analysis of Sorting Algorithms

ชื่อการทดลอง การออกแบบและการทดลองเพื่อวัดประสิทธิภาพของอัลกอริทึมการเรียงลำดับ

Experimental Performance Comparison of Five Sorting Algorithms

โจทย์สำหรับนักศึกษา

จงออกแบบและทำการทดลองเพื่อวัดประสิทธิภาพของ Sorting Algorithms จำนวน 5 วิธี ได้แก่ Bubble Sort, Insertion Sort, Selection Sort, Merge Sort และ Quick Sort โดยกำหนด Input Size และ Input Data อย่างเหมาะสม ทำการทดลองซ้ำ วัด Execution Time และคำนวณค่าเฉลี่ยของผลการทดลอง จากนั้นนำค่าเฉลี่ยที่ได้มาสร้างกราฟเพื่อเปรียบเทียบประสิทธิภาพของ Algorithms ทั้ง 5 วิธี วิเคราะห์ความสัมพันธ์ระหว่าง Input Size และ Execution Time เปรียบเทียบผลการทดลองกับ Theoretical Time Complexity และอธิบายปัจจัยที่ทำให้เกิดผลลัพธ์ดังกล่าว

ให้นำผลการทดลองทั้งหมดไปจัดทำเป็นรายงานในรูปแบบบทความวิชาการ (ใช้โปรแกรม LaTeX) โดยรายงานต้องประกอบด้วย Title, Authors, Abstract, Keywords, Introduction, Literature Review, Experimental Methodology, Results and Discussion, Conclusion และ References ตามรูปแบบ IEEE

ชื่อผู้ร่วมงาน

ลำดับ	ชื่อ-นามสกุล	รหัสนักศึกษา
1	_____	_____
2	_____	_____
3	_____	_____

กลุ่ม: _____

วันที่ทำการทดลอง: _____

อาจารย์ผู้สอน: _____

Abstract

การทดลองนี้มีวัตถุประสงค์เพื่อให้ นักศึกษาออกแบบและดำเนินการทดลองเพื่อศึกษาประสิทธิภาพของอัลกอริทึมการเรียงลำดับข้อมูล 5 วิธี ได้แก่ Bubble Sort, Insertion Sort, Selection Sort, Merge Sort และ Quick Sort โดยนักศึกษาจะต้องนำความรู้เกี่ยวกับการวิเคราะห์ประสิทธิภาพของอัลกอริทึมและ Time Complexity มาออกแบบการทดลองที่สามารถเปรียบเทียบประสิทธิภาพของแต่ละอัลกอริทึมได้อย่างเป็นระบบ การทดลองจะพิจารณาขนาดของข้อมูลที่แตกต่างกันและเวลาในการทำงานของแต่ละอัลกอริทึมซ้ำหลายครั้ง จากนั้นนำผลการทดลองมาคำนวณค่าเฉลี่ยและนำเสนอในรูปแบบตารางและกราฟ เพื่อวิเคราะห์ความสัมพันธ์ระหว่างขนาดข้อมูลกับเวลาในการประมวลผล ตลอดจนเปรียบเทียบผลการทดลองกับ Time Complexity ทางทฤษฎีของแต่ละอัลกอริทึม

ผลลัพธ์ของ Lab นี้จะต้องถูกนำไปจัดทำเป็นรายงานในรูปแบบบทความวิชาการ โดยเน้นการนำเสนอหลักฐานจากการทดลอง การวิเคราะห์ผล และการอภิปรายความสัมพันธ์ระหว่างทฤษฎีและผลการทดลองจริง ทั้งนี้ การวิเคราะห์เชิงประจักษ์มีความสำคัญเนื่องจาก Big-O Analysis สามารถแสดงอัตราการเติบโตของเวลาในการทำงานได้ แต่ไม่สามารถบอกความแตกต่างเชิงปฏิบัติระหว่างอัลกอริทึมที่มี Complexity ใกล้เคียงกันได้ทั้งหมด

Keywords: Sorting Algorithms, Algorithm Analysis, Time Complexity, Execution Time, Empirical Evaluation

1. บทนำ (Introduction)

การเรียงลำดับข้อมูล หรือ **Sorting** เป็นหนึ่งในปัญหาพื้นฐานที่สำคัญของวิทยาการคอมพิวเตอร์ และเป็นส่วนประกอบของระบบประมวลผลข้อมูลและอัลกอริทึมจำนวนมาก การเลือกอัลกอริทึมที่เหมาะสมสามารถส่งผลต่อประสิทธิภาพของโปรแกรม โดยเฉพาะเมื่อขนาดของข้อมูลเพิ่มขึ้น

ในรายวิชา CPE231 Algorithm Design นักศึกษาได้เรียนรู้การวิเคราะห์ประสิทธิภาพของอัลกอริทึมในเชิงทฤษฎี เช่น Big-O Notation และ Time Complexity อย่างไรก็ตาม การวิเคราะห์เชิงทฤษฎีเพียงอย่างเดียวไม่สามารถตอบคำถามได้ทั้งหมดว่า

“อัลกอริทึมใดทำงานได้เร็วที่สุดในสถานการณ์จริง?”

ตัวอย่างเช่น Bubble Sort, Insertion Sort และ Selection Sort มี Average/Worst-Case Complexity โดยทั่วไปอยู่ในระดับ ($O(n^2)$) ขณะที่ Merge Sort มี Complexity ($O(n \log n)$) และ Quick Sort มี Average-Case Complexity ($O(n \log n)$) แต่สามารถมี Worst Case เป็น ($O(n^2)$) ได้ ทั้งนี้ประสิทธิภาพจริงยังขึ้นอยู่กับ Implementation, Input Data และสภาพแวดล้อมของการทดลองด้วย

งานศึกษาด้าน Empirical Analysis แสดงให้เห็นว่าการทดลองจริงสามารถช่วยแยกความแตกต่างของอัลกอริทึมที่มี Asymptotic Complexity เดียวกัน และช่วยให้เห็นผลของขนาดและรูปแบบข้อมูลที่ Big-O เพียงอย่างเดียวไม่สามารถอธิบายได้ทั้งหมด

ดังนั้น Lab นี้จึงให้นักศึกษาเปลี่ยนจากการเรียนรู้ว่า

“Algorithm มี Complexity เท่าใด?”

ไปสู่คำถามว่า

“Algorithm ทำงานจริงอย่างไร และเราสามารถพิสูจน์ประสิทธิภาพของมันด้วยข้อมูลจากการทดลองได้อย่างไร?”

1.1 วัตถุประสงค์ของการทดลอง

เมื่อสิ้นสุดการทดลอง นักศึกษาสามารถ:

1. อธิบายหลักการทำงานของ Sorting Algorithms ทั้ง 5 วิธี
2. วิเคราะห์ Time Complexity ของแต่ละ Algorithm
3. ออกแบบการทดลองเพื่อเปรียบเทียบ Algorithm อย่างเป็นระบบ
4. Implement Sorting Algorithms ทั้ง 5 วิธี
5. วัด Execution Time ของ Algorithm
6. ทำการทดลองซ้ำและคำนวณค่าเฉลี่ย
7. นำเสนอผลการทดลองด้วยตารางและกราฟ
8. วิเคราะห์ความสัมพันธ์ระหว่าง Input Size และ Execution Time
9. เปรียบเทียบผลการทดลองกับ Theoretical Time Complexity
10. อภิปรายปัจจัยที่ส่งผลต่อประสิทธิภาพของ Algorithm
11. เขียนผลการทดลองในรูปแบบ Academic Paper

1.2 คำถามหลักของการทดลอง

นักศึกษาต้องใช้ในการทดลองเพื่อค้นหาคำตอบของคำถามต่อไปนี้

RQ1: เมื่อขนาดข้อมูลเพิ่มขึ้น เวลาในการทำงานของ Sorting Algorithms แต่ละวิธีเปลี่ยนแปลงอย่างไร?

RQ2: Algorithm ใดมีประสิทธิภาพดีที่สุดจากผลการทดลอง?

RQ3: Algorithm ใดมีประสิทธิภาพต่ำที่สุด?

RQ4: ผลการทดลองสอดคล้องกับ Theoretical Time Complexity หรือไม่?

RQ5: เหตุใด Algorithms ที่มี Time Complexity เหมือนกันจึงอาจมี Execution Time แตกต่างกัน?

RQ6: ขนาดของข้อมูลมีผลต่อความแตกต่างของประสิทธิภาพระหว่าง Algorithms มากน้อยเพียงใด?

2. ทบทวนวรรณกรรม (Literature Review)

นักศึกษาต้องศึกษาหลักการทำงานและ Complexity ของ Sorting Algorithms ทั้ง 5 วิธีจากตำราและแหล่งอ้างอิงทางวิชาการ

2.1 Bubble Sort

Bubble Sort ทำการเปรียบเทียบข้อมูลที่อยู่ติดกันและสลับตำแหน่งเมื่อข้อมูลอยู่ในลำดับที่ไม่ถูกต้อง

โดยทั่วไป:

[Best = $O(n)$]

[Average = $O(n^2)$]

[Worst = $O(n^2)$]

ทั้งนี้ Best Case ($O(n)$) จะเกิดขึ้นเมื่อใช้ Implementation ที่มีการตรวจสอบว่ามีการ Swap เกิดขึ้นหรือไม่

2.2 Insertion Sort

Insertion Sort แบ่งข้อมูลเชิงแนวคิดออกเป็นสองส่วนที่เรียงแล้วและส่วนที่ยังไม่ได้เรียง จากนั้นนำข้อมูลแต่ละตัวไปแทรกในตำแหน่งที่เหมาะสม

[Best = $O(n)$]

[Average = $O(n^2)$]

[Worst = $O(n^2)$]

Insertion Sort มีจุดเด่นเมื่อข้อมูลมีขนาดเล็กหรือมีลักษณะเกือบเรียงลำดับอยู่แล้ว

2.3 Selection Sort

Selection Sort ค้นหาสมาชิกที่มีค่าต่ำที่สุดจากส่วนที่ยังไม่ได้เรียง แล้วนำมาไว้ในตำแหน่งที่ถูกต้อง

[Best = $O(n^2)$]

[Average = $O(n^2)$]

[Worst = $O(n^2)$]

2.4 Merge Sort

Merge Sort ใช้แนวคิด **Divide and Conquer** โดยแบ่งข้อมูลออกเป็นส่วนย่อย เรียงข้อมูลแต่ละส่วน และนำกลับมารวมกัน

[Best = Average = Worst = $O(n \log n)$]

Merge Sort ต้องใช้พื้นที่เพิ่มเติมสำหรับการ Merge โดยทั่วไปอยู่ในระดับ ($O(n)$)

2.5 Quick Sort

Quick Sort ใช้แนวคิด Divide and Conquer เช่นเดียวกับ Merge Sort แต่ใช้ Pivot เพื่อแบ่งข้อมูลออกเป็นส่วนย่อย

[Best = $O(n \log n)$]

[Average = $O(n \log n)$]

[Worst = $O(n^2)$]

ประสิทธิภาพของ Quick Sort ขึ้นอยู่กับวิธีการเลือก Pivot และลักษณะของข้อมูล Input

2.6 ตารางเปรียบเทียบเชิงทฤษฎี

ก่อนทำการทดลอง นักศึกษาต้องจัดทำตารางต่อไปนี้

Algorithm	Best Case	Average Case	Worst Case
Bubble Sort			
Insertion Sort			
Selection Sort			
Merge Sort			
Quick Sort			

จากนั้นให้อธิบายว่า จาก Complexity ทางทฤษฎี นักศึกษาคาดว่า Algorithm ใดจะมีประสิทธิภาพดีที่สุดเมื่อ (n) มีขนาดใหญ่ และเพราะเหตุใด?

3. วิธีการทำการทดลอง (Experimental Methodology)

3.1 หลักการทดลอง

นักศึกษาต้องออกแบบการทดลองเพื่อเปรียบเทียบ Execution Time ของ Sorting Algorithms ทั้ง 5 วิธี ภายใต้เงื่อนไขที่เหมือนกัน การทดลองต้องมีหลักสำคัญคือ

Same Input — Same Environment — Same Measurement Method

เพื่อให้ผลการเปรียบเทียบมีความน่าเชื่อถือ

3.2 Algorithms ที่ต้องทดลอง

นักศึกษาต้อง Implement:

1. Bubble Sort
2. Insertion Sort
3. Selection Sort
4. Merge Sort
5. Quick Sort

ไม่อนุญาตให้ใช้ Built-in Sorting Function เป็น Algorithm หลักของการทดลอง

3.3 ขนาดข้อมูล

กำหนดให้ทดลองอย่างน้อย 6 ขนาด เช่น [$n = 1000, 5000, 10,000, 50,000, 100,000, 200,000$]

หากเครื่องคอมพิวเตอร์สามารถรองรับได้ ให้นักศึกษาเพิ่ม: [50,000, 200,000]

ทั้งนี้ต้องคำนึงถึงเวลาในการทำงานของ ($O(n^2)$) Algorithms ด้วย

3.4 Input Data

ให้ใช้ข้อมูลประเภท Integer และสร้าง Dataset ด้วยวิธีเดียวกันสำหรับทุก Algorithm

อย่างน้อยต้องมี:

Dataset 1: Random Data

ข้อมูลจำนวนเต็มที่สร้างขึ้นแบบสุ่ม

Dataset 2: Sorted Data

ข้อมูลที่เรียงจากน้อยไปมากอยู่แล้ว

Dataset 3: Reverse-Sorted Data

ข้อมูลที่เรียงจากมากไปน้อย

การใช้ Input Pattern ที่แตกต่างกันช่วยให้เห็นผลของลักษณะข้อมูลต่อประสิทธิภาพของ Algorithm

โดยงานวิจัยเชิงประจักษ์ล่าสุดก็รายงานว่า Input Pattern สามารถส่งผลต่อ Runtime ของ Sorting Algorithms โดยเฉพาะ Insertion และ Quick Sort

3.5 ตัวแปรในการทดลอง

ตัวแปรต้น

- Sorting Algorithm
- Input Size
- Input Pattern

ตัวแปรตาม

- Execution Time

ตัวแปรควบคุม

- Programming Language
- Compiler/Interpreter
- Computer
- Operating System
- Dataset
- Random Seed
- Timing Method

3.6 การวัดเวลา

ให้นักศึกษาวัดเฉพาะเวลาที่ใช้ในการ Sort

ไม่รวม:

- การสร้าง Dataset
- การอ่านข้อมูล
- การแสดงผล
- การสร้างกราฟ

หลักการวัด:

$$[T = T_{\text{end}} - T_{\text{start}}]$$

โครงสร้าง:

Generate Dataset



Copy Dataset



Start Timer



Sorting Algorithm



Stop Timer



Record Execution Time

3.7 การทดลองซ้ำ

แต่ละเงื่อนไขต้องทดลองซ้ำอย่างน้อย [R=10] ครั้ง เช่น Quick Sort + Random Data + (n=10,000) ต้องวัดอย่างน้อย 10 ครั้ง

3.8 ค่าเฉลี่ย

เมื่อได้ผลการทดลอง [T₁, T₂, ..., T_R]

ให้คำนวณค่าเฉลี่ย [$T = \frac{1}{R} \sum_{i=1}^R T_i$]

ค่าเฉลี่ยนี้จะถูกนำไปใช้สร้างกราฟเปรียบเทียบ

4. ผลการทดลองและการอธิบายผล (Experimental Results and Discussion)

4.1 ตารางผลการทดลอง

นักศึกษาต้องจัดทำตาราง Raw Data เช่น

Algorithm							
m	Input Size	Trial 1	Trial 2	Trial 3	...	Trial 10	Average
Bubble	100						
Insertion	100						
Selection	100						
Merge	100						
Quick	100						

4.2 ตารางสรุปผล

ให้นักศึกษาสร้างตารางจาก **Average Execution Time**

Input Size (n)	Bubble	Insertion	Selection	Merge	Quick
100					
500					
1,000					
5,000					
10,000					
20,000					

หน่วยต้องระบุอย่างชัดเจน เช่น **milliseconds (ms)**

4.3 การสร้างกราฟ

นักศึกษาต้องนำค่าเฉลี่ยของการทดลองทั้งหมดมาสร้างกราฟ

Figure 1: Random Data

กราฟแสดง

[Input Size vs. Average Execution Time]

โดยแสดงเส้นของ Sorting Algorithms ทั้ง 5 วิธี

Figure 2: Sorted Data

แสดงการเปรียบเทียบ Execution Time เมื่อ Input Data เรียงลำดับอยู่แล้ว

Figure 3: Reverse-Sorted Data

แสดงการเปรียบเทียบ Execution Time เมื่อ Input Data เรียงจากมากไปน้อย

4.4 การอธิบายกราฟ

นักศึกษาต้องไม่เพียงแสดงกราฟ แต่ต้องอธิบายสิ่งที่สังเกตได้จากกราฟ

ต้องตอบอย่างน้อย:

1. Algorithm ใดเร็วที่สุด?
2. Algorithm ใดช้าที่สุด?
3. เมื่อ (n) เพิ่มขึ้น Algorithm ใดมี Runtime เพิ่มขึ้นเร็วที่สุด?
4. จุดใดที่เห็นความแตกต่างระหว่าง Algorithms ชัดเจน?
5. ผลที่ได้สอดคล้องกับ Big-O หรือไม่?

4.5 การเปรียบเทียบกับ Time Complexity

ให้นักศึกษานำผลการทดลองมาเปรียบเทียบกับทฤษฎี

ตัวอย่างประเด็นที่ต้องวิเคราะห์:

กลุ่ม ($O(n^2)$)

- Bubble Sort
- Insertion Sort
- Selection Sort

กลุ่ม ($O(n \log n)$)

- Merge Sort
- Quick Sort — Average Case

นักศึกษาต้องอธิบายว่า เมื่อ (n) เพิ่มขึ้น เหตุใดกลุ่ม ($O(n^2)$) จึงมีแนวโน้มใช้เวลามากกว่ากลุ่ม ($O(n \log n)$)

งานศึกษาทาง Empirical Analysis พบแนวโน้มดังกล่าวจากการทดลองจริง และชี้ว่า Experimental Evaluation ช่วยให้เห็นความแตกต่างที่ Asymptotic Analysis เพียงอย่างเดียวไม่สามารถบอกได้

4.6 การอภิปรายผล (Discussion)

นักศึกษาต้องอภิปรายผลโดยใช้ข้อมูลจากการทดลอง ไม่ใช่เพียงอ้างทฤษฎี

ควรพิจารณาปัจจัย เช่น:

- Time Complexity
- Input Size
- Input Pattern
- Implementation
- Compiler
- CPU
- Memory
- Cache
- Recursion
- Function-call overhead

ตัวอย่างคำถาม:

หาก Quick Sort และ Merge Sort ต่างมี Average Complexity เป็น $O(n \log n)$ เหตุใด Execution Time จึงอาจไม่เท่ากัน?

และ

เหตุใด Insertion Sort อาจทำงานได้ดีเมื่อข้อมูลมีการเรียงลำดับอยู่แล้ว?

4.7 ข้อจำกัดของการทดลอง

นักศึกษาต้องระบุน้อย 2 ข้อจำกัด เช่น

- การทดลองใช้ Computer เพียงเครื่องเดียว
- Programming Language มีผลต่อ Runtime
- Background Processes อาจส่งผลต่อเวลา
- Timer มีข้อจำกัดด้าน Resolution
- Dataset มีขนาดจำกัด
- Implementation ของ Algorithm อาจแตกต่างกัน

5. สรุปผลการทดลอง (Conclusion)

ในส่วนนี้ให้นักศึกษาสรุปผลจากข้อมูลที่ได้จริง

ต้องตอบประเด็นต่อไปนี้:

1. Algorithm ใดมีประสิทธิภาพดีที่สุดจากการทดลอง?
2. Algorithm ใดมีประสิทธิภาพต่ำที่สุด?
3. Input Size มีผลต่อ Runtime อย่างไร?
4. Input Pattern มีผลต่อ Runtime หรือไม่?
5. ผลการทดลองสอดคล้องกับ Big-O หรือไม่?
6. ผลการทดลองใดน่าสนใจหรือแตกต่างจากที่คาดไว้?
7. จากการทดลอง นักศึกษาจะเลือก Algorithm ใดสำหรับข้อมูลขนาดใหญ่ เพราะเหตุใด?

ข้อสำคัญ: นักศึกษาต้องไม่กำหนดคำตอบไว้ล่วงหน้า ผลสรุปต้องมาจาก Experimental Evidence ของกลุ่มตนเอง

6. รูปแบบรายงานวิชาการที่ต้องส่ง

ผลการทดลองทั้งหมดต้องนำไปจัดทำเป็น **Academic Report / Seminar Paper**

รายงานต้องประกอบด้วยส่วนต่อไปนี้

Title

ชื่อเรื่องต้องสะท้อนเนื้อหาการทดลอง เช่น

Experimental Performance Comparison of Five Sorting Algorithms

หรือ

An Empirical Study of Bubble, Insertion, Selection, Merge, and Quick Sort

Authors

ระบุ:

- ชื่อสมาชิก
- รหัสนักศึกษา
- กลุ่มเรียน

Abstract ความยาวประมาณ **150–250 words**

ต้องประกอบด้วย: **Background** → **Objective** → **Method** → **Results** → **Conclusion**

Keywords กำหนด 4–6 คำ เช่น:

Sorting Algorithms; Algorithm Analysis; Time Complexity; Execution Time; Empirical Evaluation

ส่วนที่ 1 บทนำ (Introduction)

ต้องประกอบด้วย:

1. ความสำคัญของ Sorting
2. ปัญหาด้าน Algorithm Performance
3. Motivation
4. วัตถุประสงค์
5. Research Questions
6. ขอบเขตของการทดลอง

ส่วนที่ 2 บททวนวรรณกรรม (Literature Review)

ต้องกล่าวถึง:

- Bubble Sort
- Insertion Sort
- Selection Sort
- Merge Sort
- Quick Sort
- Time Complexity
- Big-O Analysis
- Empirical Algorithm Analysis

ควรใช้ตารางและบทความวิชาการเป็นหลัก

งานวิจัยที่ศึกษา Sorting Algorithms ทั้ง 5 วิธีโดยตรงได้ใช้ Execution Time, Dataset Size และ Time Complexity เป็นตัวชี้วัดในการเปรียบเทียบเชิงทดลองเช่นกัน

ส่วนที่ 3 วิธีการทำการทดลอง (Experimental Methodology)

ต้องอธิบาย:

3.1 Hardware

- CPU
- RAM
- Operating System

3.2 Software

- Programming Language
- Version
- Compiler/Interpreter
- Development Environment

3.3 Algorithms

อธิบาย Algorithm ทั้ง 5

3.4 Dataset

อธิบาย:

- Input Size

- Data Type
- Random Data
- Sorted Data
- Reverse-Sorted Data

3.5 Experimental Procedure

อธิบายขั้นตอนการทดลอง

3.6 Performance Measurement

อธิบายวิธีวัดเวลาและการทดลองซ้ำ

3.7 Data Analysis

อธิบายการคำนวณ Average และการสร้างกราฟ

ส่วนที่ 4 ผลการทดลองและการอธิบายผล (Results and Discussion)

ต้องประกอบด้วย:

4.1 Raw Experimental Data

4.2 Average Execution Time

4.3 Graphical Comparison

4.4 Effect of Input Size

4.5 Comparison among Algorithms

4.6 Comparison with Theoretical Complexity

4.7 Discussion

4.8 Limitations

ส่วนที่ 5 สรุปผลการทดลอง (Conclusion)

สรุปผลที่สำคัญจากการทดลองและตอบ Research Questions

บรรณานุกรม (References)

ใช้รูปแบบ **IEEE Reference Style**

ควรมีแหล่งอ้างอิงอย่างน้อย **5 แหล่ง**

7. รูปแบบการส่งรายงาน

นักศึกษาต้องส่ง:

7.1 Source Code

Source Code ของ Sorting Algorithms ทั้ง 5 วิธี

7.2 Raw Data

ข้อมูลการทดลองทั้งหมด

7.3 Graphs

กราฟที่สร้างจากค่าเฉลี่ยของผลการทดลอง

7.4 Academic Report

รายงานฉบับสมบูรณ์ในรูปแบบ PDF

8. Rubric การประเมินผล

คะแนนเต็ม **100 คะแนน**

รายการประเมิน	คะแนน
1. การ Implement Algorithms ทั้ง 5 วิธี	15
2. การออกแบบการทดลอง	15
3. การเก็บข้อมูลและความครบถ้วน	10
4. การคำนวณค่าเฉลี่ยและวิเคราะห์ข้อมูล	10
5. การสร้างกราฟและการนำเสนอข้อมูล	10
6. การนำเสนอผลการทดลอง	10
7. การอธิบายและอภิปรายผล	15
8. รูปแบบและการเขียนบทความวิชาการ	5
9. การอ้างอิงและบรรณานุกรม	5
10. คุณภาพ Code และความสามารถในการทำซ้ำการทดลอง	5
รวม	100

9. เกณฑ์คุณภาพของรายงาน

ระดับ Excellent

นักศึกษาสามารถ:

- Implement Algorithms ได้ถูกต้อง
- ออกแบบการทดลองได้อย่างเป็นระบบ
- ใช้ Dataset เดียวกันในการเปรียบเทียบ
- ทดลองซ้ำอย่างเพียงพอ
- คำนวณค่าเฉลี่ยถูกต้อง
- สร้างกราฟที่เหมาะสม
- วิเคราะห์ผลโดยใช้ข้อมูลจริง
- เชื่อมโยง Experimental Result กับ Big-O
- อธิบายเหตุผลของผลลัพธ์
- อภิปรายข้อจำกัด
- เขียนในรูปแบบ Academic Paper

ระดับ Good

การทดลองและรายงานถูกต้องเป็นส่วนใหญ่ แต่การวิเคราะห์ยังไม่ลึกมาก

ระดับ Fair

สามารถดำเนินการทดลองได้ แต่ยังมีข้อผิดพลาดด้าน Data Collection, Visualization หรือ Discussion

ระดับ Poor

การทดลองไม่ครบ ข้อมูลไม่น่าเชื่อถือ หรือไม่สามารถเชื่อมโยงผลการทดลองกับทฤษฎีได้

10. ข้อกำหนดสำคัญในการทดลอง

นักศึกษาต้องตระหนักว่า การทดลองที่ดีไม่ได้หมายถึงการได้ผลลัพธ์ตรงกับทฤษฎี

หากผลการทดลองแตกต่างจากสิ่งที่คาดการณ์ไว้ นักศึกษาต้องวิเคราะห์ว่าเหตุใดจึงเกิดความแตกต่าง

ตัวอย่างเช่น

[]

ไม่ได้หมายความว่าผลการทดลองผิดเสมอไป แต่อาจเกิดจาก:

[]

ดังนั้น สิ่งที่สำคัญที่สุดของ Lab นี้คือ ความสามารถในการอธิบาย Experimental Evidence อย่างมีเหตุผล

11. ผลลัพธ์การเรียนรู้ที่คาดหวัง

เมื่อทำ Lab 2 เสร็จ นักศึกษาควรเปลี่ยนจากการมอง Algorithm ในลักษณะ:

“Bubble Sort = $O(n^2)$ ”

“Merge Sort = $O(n \log n)$ ”

ไปสู่การคิดในลักษณะ:

“เราจะออกแบบการทดลองอย่างไรเพื่อพิสูจน์ว่า Algorithm เหล่านี้มีพฤติกรรมแตกต่างกันอย่างไร?”

และสามารถดำเนินกระบวนการ

[]

ได้ด้วยตนเอง

12. บรรณานุกรมเบื้องต้นสำหรับนักศึกษา

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[2] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Boston, MA, USA: Addison-Wesley, 2011.

[3] J. Kleinberg and É. Tardos, *Algorithm Design*. Boston, MA, USA: Pearson, 2006.

[4] A. N. Frak, M. Z. Saringat, Y. A. Prasetyo, A. Mustapha, and H. Aman, “Comparison study of sorting techniques in static data structure,” *International Journal of Integrated Engineering*.

[5] D. S. Sutanto, C. Kirana, and D. Wahyuningsih, “Analisis Efisiensi Waktu Bubble, Insertion, Merge, dan Quick Sort Menggunakan Python,” *METIK Jurnal*, vol. 9, no. 1.

[6] “Empirical Comparison of Sorting Algorithms,” *Data Structures and Algorithms*, Chalmers University of Technology.

[7] “A systematic analysis on performance and computational complexity of sorting algorithms,” *Discover Computing*, Springer, 2025.